听·见：视听融合监测节点

——基于 PYNQ-Z2 的鸟类声视协同监测系统（以仙八色鸫为例）

摘要

针对鸟类生态观测中人工值守耗时长、连续录像冗余大的问题，本作品设计并实现了一套基于 AMD PYNQ-Z2 的鸟类声视协同监测系统，以国家二级保护动物仙八色鸫（Pitta nympha）为观测对象，构建“声学常驻监听—视觉按需确认—事件化记录”的自动化监测流程。

节点采用软硬件协同架构，由两块可独立加载的 PL 设计构成。采集设计以自研 I^2S 核接收 INMP441 的 32 kHz/24 bit 音频流，经 AXI-DMA 成包写入 DDR，仅占用 LUT 3.15%、BRAM 0.71%；加速设计集成基于 Vitis HLS 的定点 GEMM 核，采用 16×16 分块与流水线结构，以 ap_fixed<16,4>（Q12）完成乘加、偏置与 ReLU 的融合，纯核比 numpy 快 4.64 倍，但端到端因 DMA 搬运开销较大仅 0.59 倍；正式主程序 conv1 由 PS 端 numpy 实现，GEMM 作加速能力验证。PS 侧以 numpy 实现梅尔特征与声学卷积网络推理，2 s 窗口、1.5s 步进输出鸣声置信度。

检出目标后开始抓拍、录像并写入板载存储，目标消失并达到缺席阈值后停止，录像期间暂停声学推理。全部推理与记录在板端离线完成，不依赖云端。

验证结果表明：两块 PL 设计均在 100 MHz 时钟下时序收敛（采集设计 WNS +2.029 ns、加速设计 WNS +1.520 ns，均无失败端点）；采集设计仅占用 LUT 3.15%、BRAM 0.71%、DSP 0%，加速设计占用 LUT 10.11%、BRAM 7.86%、DSP 30.45%；板上 GEMM 硬件输出与浮点参考逐点对比，最大偏差 0.001838（205824 点全部落在 0.02 容差内）。声学事件级召回 97.4%、图像独立召回 96.2%、平均响应时延约 10.5 s、可长期无人值守运行。节点以低开销的声学常驻监听触发按需视觉确认，减少无效视觉推理与冗余录像，可为林地、自然保护地等场景提供低成本、可复核的辅助监测手段。

关键词：视听融合监测；鸟类鸣声识别；PYNQ-Z2；Vitis HLS；定点 GEMM 加速

第一部分 作品概述

1.1 选题背景与问题定义

鸟类是生态系统的重要指示类群，长期、连续的种群监测是生物多样性评估的基础。传统观测依赖人工巡听与定点守候，单点单日的有效观测时间有限；被动声学记录仪虽能长时间录音，但产生的音频量远超人工筛选能力；红外相机受枝叶遮挡与触发距离限制，常出现只闻其声、难见其形的情况。无论哪种手段，把原始数据转化为可复核的观测记录仍需大量人工。

前人工作大致沿三条技术线展开。一是以自动录音单元（ARU）为载体的被动声学监测，解决了长时间无人值守录音与留存的问题[1]；二是以 BirdNET 为代表的深度鸟鸣识别，由康奈尔鸟类学实验室与凯姆尼茨工业大学联合开发，以 3 s 音频片段生成频谱图并送入卷积网络，把海量录音自动筛成物种事件[2]，但其输出仍是时间—物种记录，本身不承担影像取证；三是红外相机与视觉检测，解决了影像取证问题，工程上已广泛部署。三条线各自成熟，却分别停留在录到、识别出、拍到三个单一环节。

近年也出现了声学与视觉结合的多模态尝试，以及若干在 FPGA 上做鸟类声学识别的工作；前者多运行在通用 CPU 或手机上，或依赖云端连接，后者通常只完成声学识别，未包含视觉确认环节[3][4]。将声学发现、视觉证实、事件化留证串成闭环，并在单块 FPGA SoC 上完全离线完成、且由可编程逻辑承担音频采集与首层卷积加速验证的方案，在已检索到的工作中尚未见同样的组合。

本作品以仙八色鸫（Pitta nympha）为观测对象。该物种被列入国家二级重点保护野生动物[5]，栖息于林下、生性隐蔽，影像获取困难。这类听得到却不易看到的物种，恰好需要声学负责发现、视觉负责证实。作品把三个环节串成一条链路，全部放在一块 PYNQ-Z2 上离线完成：声学常驻监听负责发现，视觉模型按需唤醒负责证实，事件化记录负责留证。

1.2 功能与特性

节点围绕“发现—确认—记录”三个环节组织功能：

（1）常驻声学监听：INMP441 持续采集环境声，以 2 s 滑动窗口、1.5 s 步

进输出鸣声置信度，未命中阈值时不触发视觉计算；

（2）事件触发：置信度超过 0.985 阈值即生成一次观测事件，输出触发时刻与声学分数；

（3）视觉确认：事件发生后唤醒视觉模型，在固定视角下 30 s 搜索窗口内逐帧检测仙八色鸫；

（4）自动记录：检出目标后抓拍带检测框的照片并录像，连续三次未检出后结束录像并回到监听；

（5）本地留存：照片、视频与日志按事件时间戳写入板载存储，支持事后复核；

（6）资源协调：录像与转码期间通过文件锁暂停音频识别，结束后恢复，避免双进程争抢 CPU。

特性方面：全流程板端离线，不依赖网络与云端；事件驱动使视觉推理与录像只发生在有鸟时段，从源头压缩冗余数据；采集与加速解耦为两块可独立加载的设计，各自资源占用低，便于分步验证与迭代；具备 systemd 自启与异常重启能力。



图 1-1 系统运行状态机示意图

1.3 应用领域

本作品面向鸟类生态观测的高效需求，主要应用于以下场景。

（1）林地与自然保护地的长期监测：设备可长期布设于样地，自动筛出有

效观测事件，替代人工定时巡听与长时间盯守，降低野外工作量。

（2）科研样地调查与生物多样性评估：事件级记录自带时间戳与置信度，便于统计物种出现频次与日活动节律，为样地评估提供结构化数据。

（3）保护区巡护与科普教育：抓拍照片与短视频可直接用于巡护记录、公众科普与自然教育素材。

（4）可迁移的鸟情监测：更换识别模型后，可扩展至机场鸟击防范、风电场鸟类活动监测等场景。

与红外相机相比，声学通道不受枝叶遮挡影响；与纯声学记录仪相比，本节点增加了视觉确认环节，输出的影像可直接用于物种复核。二者可互补布设。

1.4 主要技术特点

（1）软硬协同的双设计：采集设计负责高带宽音频搬运，加速设计负责首层卷积，PS 负责特征、调度与决策，三者通过寄存器与数据流接口解耦，可按需加载、独立验证。

（2）事件驱动的按需唤醒：声学通道低开销常驻，视觉通道按需唤醒，使视觉推理次数与目标出现次数同阶。

（3）定点 GEMM 的流水化实现：16×16 分块覆盖任意 M，K≤16 时单块完成；权重整块驻留片上，A 分块流式读入，乘加、偏置、ReLU 与饱和在输出流水线内完成，末拍 TLAST 由全局包计数生成，保证 DMA 可反复调用。

（4）轻量的板端推理栈：声学推理依赖 numpy，视觉推理使用 ncnn，无需 PyTorch 或 ONNX Runtime。

（5）事件化记录：快照、视频与逐帧预测日志按事件归档，支持复核与统计。

1.5 主要性能指标

关键指标如表 1-1 所示，数据均来自板端实测、PC 端离线评测与 Vivado 实现报告。

表 1-1　主要性能指标

类别	指标	数值
采集	逻辑资源	LUT 3.15%、FF 2.14%、BRAM 0.71%、DSP 0
采集	时序 / 功耗	WNS 2.029 ns；1.684 W（估算，Low）
采集	音频格式	32 kHz、24 bit、单声道，2 s 窗/ 1.5 s 步进
加速	逻辑资源	LUT 10.11%、FF 7.05%、BRAM 7.86%、DSP 30.45%
加速	时序 / 功耗	WNS 1.520 ns、WHS 0.008 ns；1.741 W（估算，Medium）
加速	conv1 规模	M=12864、K=9、N=16，ap_fixed<16,4>
加速	conv1 纯核耗时	6.7 ms（ap_start→ap_done，20 次中位数）
加速	公平加速比	4.64×（纯核）/ 0.59×（端到端 52.7 ms）
加速	conv1 数值误差	0.001838（205824 点，0 点超 0.02）
识别	音频事件级召回	97.4%（阈值 0.985）/ 98.1%（0.5）
识别	音频负样本	2063 个窗口 0 误报，最高分 0.0329
识别	图像检出 / 特异性	99.2%（训练池）/ 380/380 无漏报
识别	图像 mAP	86.7%（训练池）
识别	图像独立召回	96.2%（256/266）
时延	音频单窗推理	2.075 s
时延	图像单帧推理	5.65 – 6.72 s（均值 6.17 s）
时延	声到像触发延迟	约 10.5 s
可靠性	连续运行时长	≥8h 实测连续运行（音频服务每 1800 s 自动换会话，属设计行为）

说明：图像 99.2% 与 mAP 86.7% 为训练池口径，独立测试见表 3-1；功耗为 Vivado 静态估算，非板端实测。

1.6 主要创新点

（1）声学常驻、视觉按需：视觉推理按鸣声触发，录像覆盖目标在场时段，压缩无效计算与冗余影像。

（2）采集与加速解耦：两块 overlay 分时加载、独立验证，LUT 分别占用仅 3.15% 与 10.11%。

（3）16×16 分块的定点 GEMM 闭环：流式读写、权重驻留、偏置与 ReLU 融合、末拍 TLAST 精确生成。

（4）面向 32 位 ARM 的轻量推理栈：声学用 numpy、视觉用 ncnn[6]，不依赖 PyTorch 与 ONNX Runtime。

（5）以数据定软硬划分：conv1 端到端仅 0.59 倍（DMA 占 87%），据此主程序保留 PS，避免为加速而加速。

1.7 设计流程

作品按“需求分析—数据与模型—PL 设计—PS 集成—联调实测”五个阶段推进：明确声视协同判定逻辑与资源约束；采集鸣声与图像，训练声学 CNN 与视觉检测器，分别导出 numpy 权重与 ncnn 模型；用 Vitis HLS 实现 GEMM 核并完成 C 仿真、联合仿真与 IP 打包，在 Vivado 中分别搭建采集与加速两个 Block Design；在 PYNQ 上以 Python 完成采集、调度与记录，视觉推理调用板端 C++程序；最后进行板级联调与验证。



图 1-2　系统设计流程图

第二部分　系统组成及功能说明

2.1 整体介绍

节点由声学采集、PL 加速、ARM 处理、视觉确认与记录五部分组成，总体框图如图 2-1 所示。

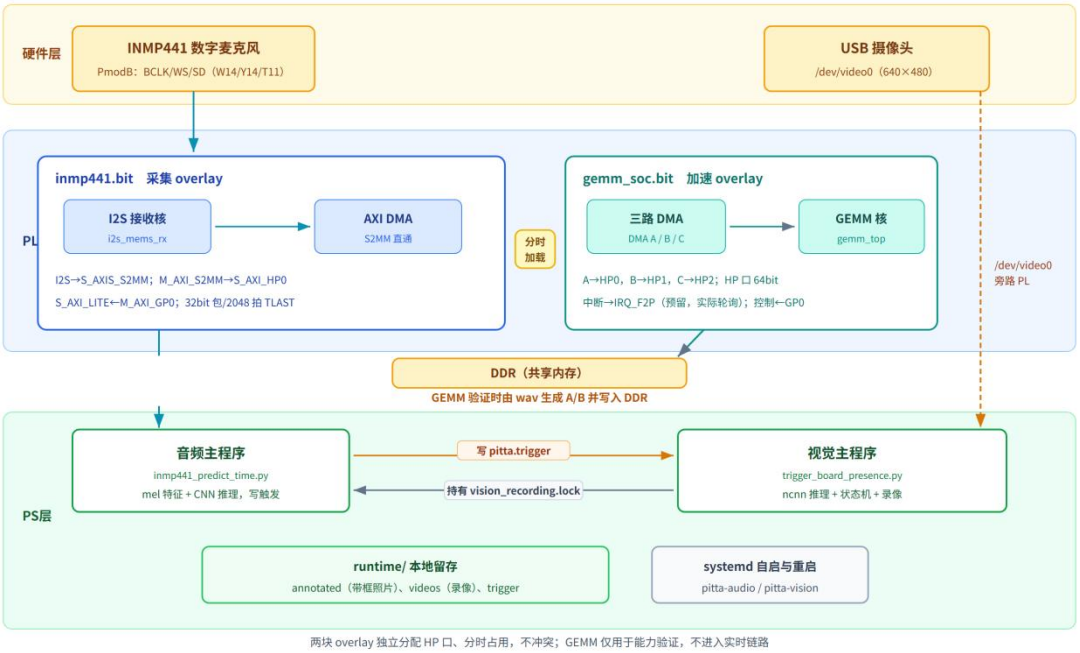


图 2-1　系统总体框图

实时链路：INMP441 输出 24 bit 音频，采集核整理为 [L,R] 交替的 32 bit 包（每包 2048 拍、1024 帧）送入 DMA；DMA 以 S2MM 直通写 DDR，音频主程序按包轮询并写入环形缓冲；每 1.5 s 取最近 2 s 音频，完成梅尔特征与 CNN 推理得到置信度；命中阈值则写出触发文件。

视觉链路：视觉主程序检测到触发文件后进入搜索，按 0.2 Hz 取帧、ncnn 推理、判定仙八色鸫；检出即录像，之后按缺席次数决定停止；录像期间持有文件锁，音频端据此暂停。

加速验证链路：另行加载 gemm_soc.bit，用 wav 回放替代麦克风，把同一段音频的 im2col 结果送入硬件 GEMM，与 numpy 逐点比对并计时。两块设计共用 PS 与 DDR，分时加载。

2.2 设计原理与功能框图

声学采集原理：采集核工作在 I^2S 主模式，BCLK 为帧率的 64 倍，WS 指示声道，SD 在上升沿按 24 bit、MSB 优先移入；核内以相位累加器从 100 MHz 分频得到 BCLK，实际帧率 32000.24 Hz，误差小于 1e-5；每帧把左右声道各符

号扩展为 32 bit、交替写入 256 深 FIFO，每 2048 拍产生一次 TLAST。

声学识别原理：2 s 音频经短时傅里叶变换（帧长 1024、帧移 320）、64 维梅尔滤波与对数归一化得到 64×201 特征图，再经三层 3×3 卷积（16/32/64 通道）、池化、4×1 平均池化、全连接与 sigmoid 输出置信度，阈值 0.985。

硬件加速原理[7]：首层卷积等价于矩阵乘——特征图按 3×3 邻域展开为 A（12864×9），BN 融合后的权重转置为 B（9×16），则 conv1 = A • B + bias，再取 ReLU。核内采用 ap_fixed<16,4>（Q12）与 32 位累加器，输出时对齐小数点、加偏置、ReLU 并饱和至 7.99976。

视觉原理：YOLOv8n[8]导出为 ncnn 模型，输入 320×320 letterbox，输出经 sigmoid 与 NMS 得到检测框；板端以 C++ 程序（2 线程）执行推理，Python 侧负责状态机与录像。

上述四部分对应的模块连接关系与数据流向见图 2-1，软硬件划分依据见 2.5。

2.3 硬件系统介绍

2.3.1 硬件整体介绍

硬件以 PYNQ-Z2 为核心，外接数字麦克风与 USB 摄像头，主要部件如表 2-1。

表 2-1　主要硬件部件

序号	部件	规格	接口	作用
1	主控板卡	PYNQ-Z2（XC7Z020-1CLG400C，512 MB DDR3）	—	PL 与 PS
2	数字麦克风	INMP441（I²S，24 bit）	PmodB	声学采集
3	摄像头	1080p USB UVC（采集 640×480、5 fps）	USB Host	视觉确认与录像
4	存储	MicroSD 卡	SD	系统、模型与记录文件
5	供电	5 V 电源	DC 输入	整机供电

2.3.2 机械设计介绍

本作品无运动机构，机械设计围绕固定安装与稳定指向展开。外壳为 200 mm × 170 mm × 70 mm 的纸盒，正面开直径 7 mm 的声孔，盒顶固定 USB 摄像头，线缆自背面开孔引出。摄像头固定于盒顶，装配时校准镜头朝向观测区，装配后不再调整，以保证视场稳定；麦克风正对声孔，以保证声学通道畅通。整机尺寸、开孔位置与装配关系见图 2-2。安装示意与实物照片见第三部分。

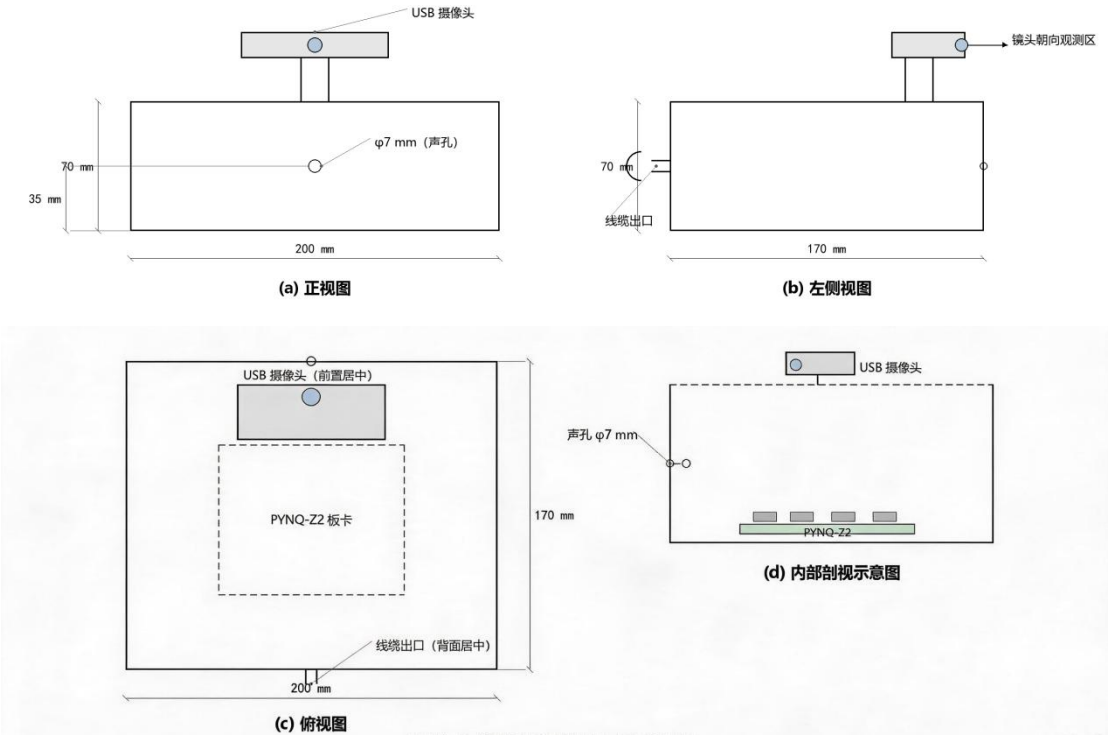


图 2-2　整机机械结构与尺寸标注

2.3.3 电路各模块介绍

系统基于 PYNQ-Z2 搭建，设计集中在接口连接、电平匹配与信号完整性。INMP441 工作于 3.3 V、标准 I²S 从模式，SCK/WS/SD 接 PmodB 前三针，L/R 接地选左声道；SD 线约 100 kΩ 下拉，VDD 就近 0.1 μF 去耦。引脚如表 2-2。

表 2-2　INMP441 与 PmodB 引脚分配

INMP441 引脚	PmodB 针脚	FPGA 引脚	方向	说明
SCK（BCLK）	1	W14	输出	I²S 位时钟
WS（LRCLK）	2	Y14	输出	声道选择，32 kHz

INMP441 引脚	PmodB 针脚	FPGA 引脚	方向	说明
SD（DOUT）	3	T11	输入	串行数据，弱下拉
VDD	6 / 12	3.3 V	电源	禁止接 5 V
GND	5 / 11	GND	地	与板卡共地
L/R	—	GND	—	选左声道

摄像头经板载 USB Host 供电与通信，UVC 免驱，默认 /dev/video0；整机 5 V 供电，板内转 3.3 V。

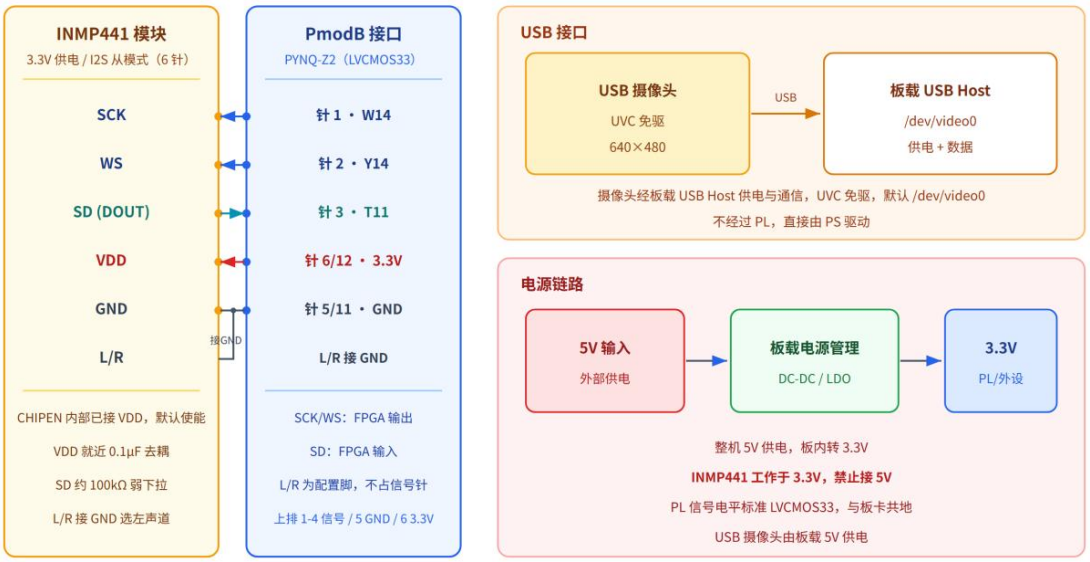


图 2-3　系统接线与接口示意

2.4 软件系统介绍

2.4.1 软件整体介绍

软件运行于 PYNQ 的 Linux 环境，用 Python 3 编写，分应用层、算法层、驱动层与硬件层（图 2-4）。应用层是音频主程序与视觉主程序；算法层为 numpy 声学推理与 ncnn 视觉推理；驱动层封装 Overlay、DMA 与 MMIO[9]；硬件层为两块 PL 设计。

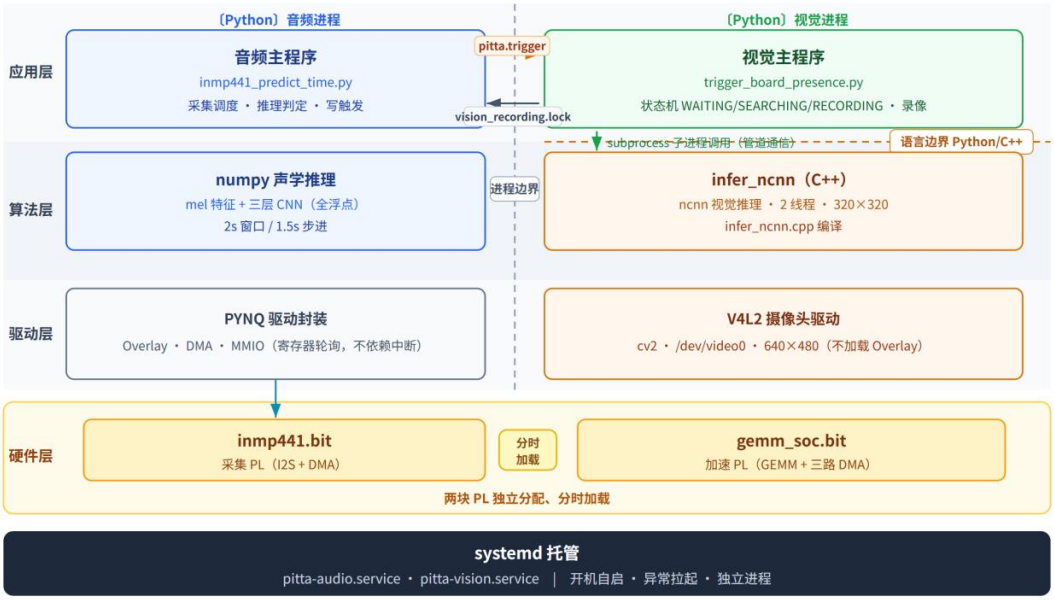


图 2-4　软件分层结构图

两个主程序由 systemd 托管（pitta-audio.service、pitta-vision.service），各自独立进程、开机自启、异常拉起，通过 runtime/trigger 下的触发文件与文件锁协作：音频端命中写触发文件；视觉端读触发并进入搜索；视觉端录像期间持有 vision_recording.lock，音频端检测到锁即暂停推理。所有 PL 访问用寄存器轮询，不依赖中断。

2.4.2 软件各模块介绍

（1）音频主程序 inmp441_predict_time.py：循环接收 DMA 包，取左声道归一化后写入环形缓冲；每 1.5 s 取 2 s 窗做 numpy 推理；命中阈值写出 JSON 触发（含北京时间、Unix 时间、分数、窗口区间、event_id），并通过 --audio-pause-file 指定录像锁路径，检测到锁即暂停推理。接口见表 2-3。

表 2-3　音频主程序接口

项目	内容
输入	I^2S 数据流（32 bit×2048/包）；参数 −-threshold / --hop / --min-hits / --cooldown / --release-threshold / --release-hits / --audio-pause-file 等
输出	runtime/trigger/pitta.trigger（JSON）；控制台分数日志
关键寄存器	CR=0x30、SR=0x34、DA_L=0x48、DA_H=0x4C、LEN=0x58
异常处理	SR 错误位或超时则重采，不中断主循环

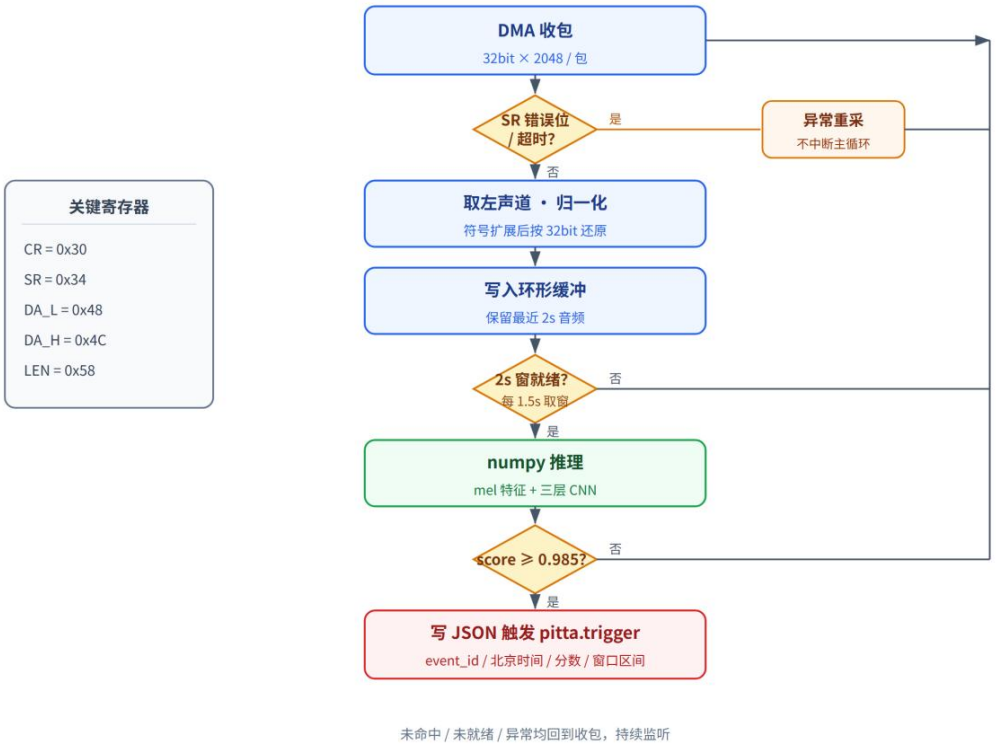


图 2-5　音频采集与触发流程图

（2）声学推理 audio_numpy.py：纯 numpy 复现训练时的前向，含 STFT、64 维梅尔、三层卷积、池化与全连接，输出 0~1 分数；已与训练框架结果做过 PC 端离线比对，分数一致。

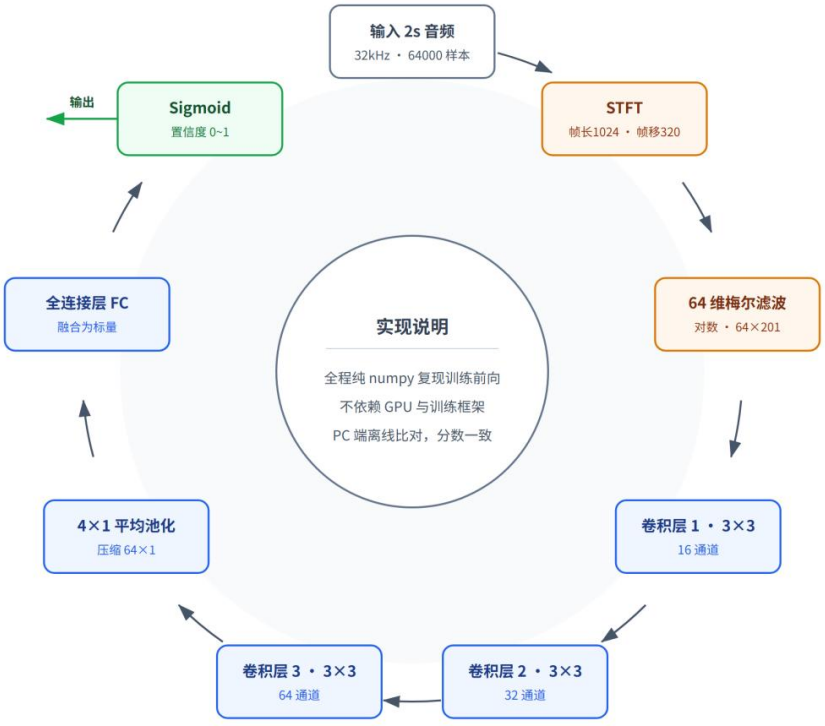


图 2-6　声学推理流程图

（3）视觉主程序 trigger_board_presence.py：状态机为等待—搜索—录像—收尾。等待时监听触发文件；搜索时限 30 s，按 0.2 Hz 取帧；检出后开始录像并持续推理，连续 3 次未检出即停止，随后转码为 H.264 MP4。

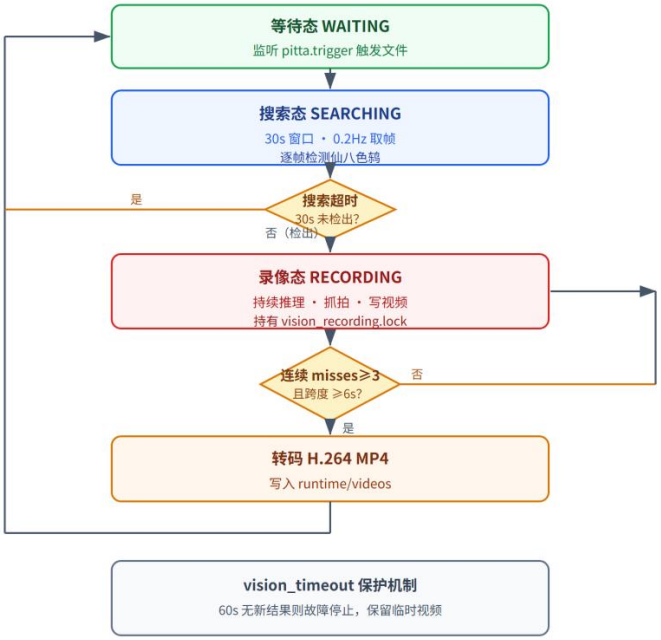


图 2-7　视觉状态机与存在性录像流程图

（4）ncnn 推理程序 infer_ncnn：C++ 编写、2 线程、输入 320×320，输出框与置信度；由 build_board.sh 在板端编译。



图 2-8　ncnn 推理调用流程图

（5）录像锁 av_recording_lock.py：基于 flock 的互斥锁，录像与转码期间持有；音频端轮询锁文件，存在即暂停推理。

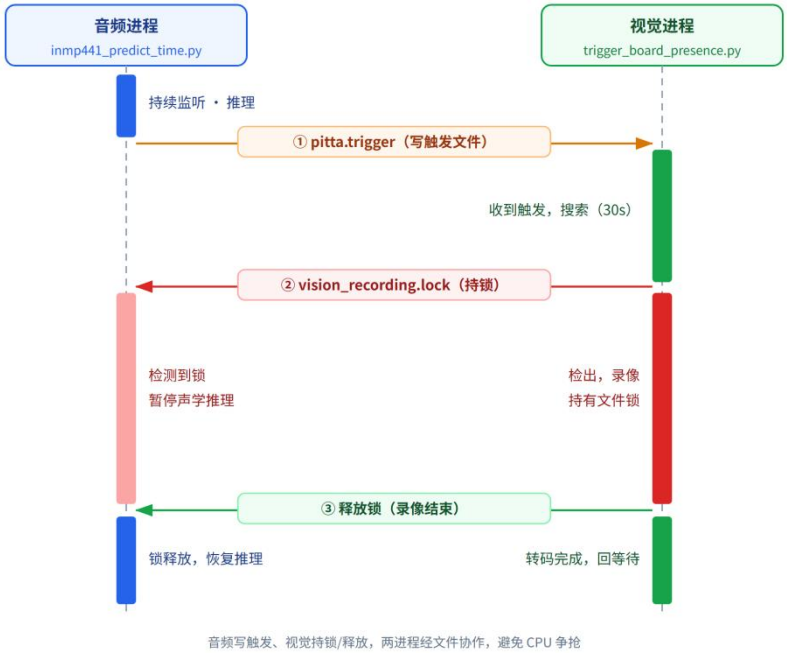


图 2-9 双进程联动时序图

（6）记录模块：快照 frame_*.jpg 与预测日志 frame_*.json（时间戳、帧号、框坐标、置信度、推理耗时）写入 runtime/annotated；视频写入 runtime/videos。

2.5 软硬件划分与接口设计

划分依据三条：其一是数据率与实时性，32 kHz×32 bit 的连续流约 2 Mbit/s，若由 PS 逐样本搬运会持续占用 A9，故把 I^2S 与 DMA 放在 PL；其二是计算特征，首层卷积 im2col 后是 12864×9 的规整矩阵，无跨通道依赖，适合流水化，而 conv2、conv3 的 K 为 144、288，超出当前核 K≤16 的限制，仍留在 PS；其三是实测数据，conv1 端到端仅 0.59×（DMA 占 87%），因此正式主程序保留 numpy，GEMM 作加速能力验证（瓶颈分析见 3.4）。

两块设计分时加载，互不叠加资源占用：采集 overlay 只用 3.15% LUT、0 DSP，加速 overlay 的 30.45% DSP 仅在验证时占用。两者的接口如表 2-4。

表 2-4　两块 PL 设计的接口划分

设计	接口	类型	规格	用途
采集	AXI-Lite	控制	32 bit	DMA 寄存器
采集	I²S→DMA	AXI-Stream	32 bit	音频数据流
采集	M_AXI_S2MM	存储	26 bit 长度	写 DDR
加速	AXI-Lite	控制	32 bit	3 个 DMA 与 GEMM 寄存器
加速	dma_A/B→gemm	AXI-Stream	16 bit	矩阵 A、B
加速	gemm→dma_C	AXI-Stream	16 bit	结果 C
加速	M_AXI_MM2S/S2MM	存储	23 bit 长度	HP0/HP1/HP2

据此，PL 承担全部硬实时数据通路：自研 I²S 接收核在 100 MHz 下自行分频产生 2.048 MHz BCLK，完成 24 bit 采样、左右声道成包与 2048 拍 TLAST 标记；AXI-DMA 以直通模式把 256 kB/s 音频流稳定写入 DDR，不占用 A9 的搬运时间。另一块 PL 设计是 HLS 定点 GEMM 核，独立完成 conv1 的 12864×9×16 乘加（纯核 6.7 ms，与浮点参考最大偏差 0.001838）。PS 只保留可变的决策与后处理——特征、后段网络、状态机与录像。两块 overlay 通过 AXI-Lite 与 AXI-Stream 交互，软件按固定偏移读写。

第三部分　完成情况及性能参数

3.1　整体介绍

节点由一块 PYNQ-Z2、一个 INMP441 数字麦克风与一个 USB 摄像头组成，封装于一体式外壳内，摄像头以强力胶固定于顶部、麦克风经声孔与外界连通，整机无运动部件，可按固定视角长期布设。两块 PL 设计均已完成实现与时序收敛，软件以 pitta-audio、pitta-vision 两个 systemd 服务常驻运行，开机自启、异常拉起。已完成端到端联动实测：一次完整闭环为鸟鸣触发→视觉搜索→

检出→录像→连续缺席 3 次停录→转码 H.264 MP4，全流程无需人工干预。整机外观见图 3-1、图 3-2，内部装配见图 3-3。



图 3-1　整机正面实物照片



图 3-2　整机斜 45°实物照片

3.2　工程成果

3.2.1 机械成果

按 2.3.2 的结构完成装配，机械外观成果见图 3-1、图 3-2，板卡与外设的连接见图 3-3：摄像头经水平校准后固定；麦克风正对声孔；线缆自背面开

孔引出；内部走线以扎带固定。强力胶固定方式在长期布设下的可靠性有限，后续拟改用 3D 打印支架。

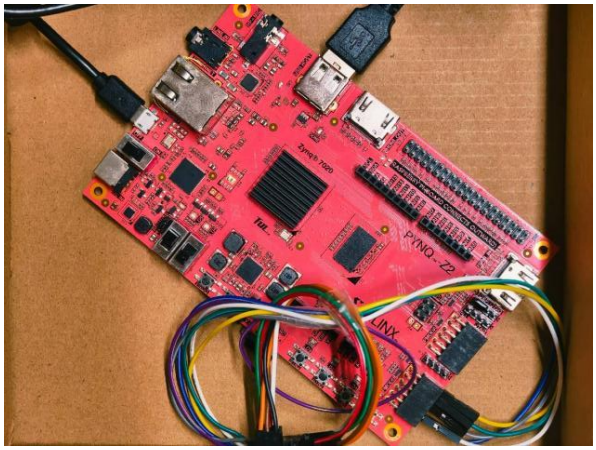


图 3-3 板卡与外设的连接

3.2.2 电路与 PL 设计成果

PL 侧完成两块 Block Design 与两个自研 IP：I²S 接收核以 RTL 实现，含 20 位相位累加分频、左右声道成包、256 深 FIFO 与 2048 拍 TLAST 生成；GEMM 加速核由 Vitis HLS 实现，含 16×16 分块、Q12 定点、偏置与 ReLU 融合、末拍 TLAST 修正。两块设计均在 100 MHz 下时序收敛（WNS 2.029 / 1.520 ns），资源占用 3.15% 与 10.11% LUT，已生成 bit/hwh 并完成板级验证，报告见 build/ 与 build/gemm/。PmodB 接线见图 3-4，两块 BD 框图见图 3-5（音频采集）与图 3-6（GEMM 加速）。

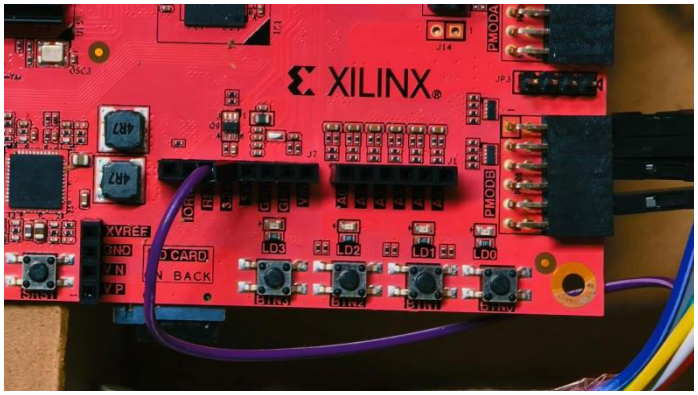


图 3-4 PmodB 接线特写

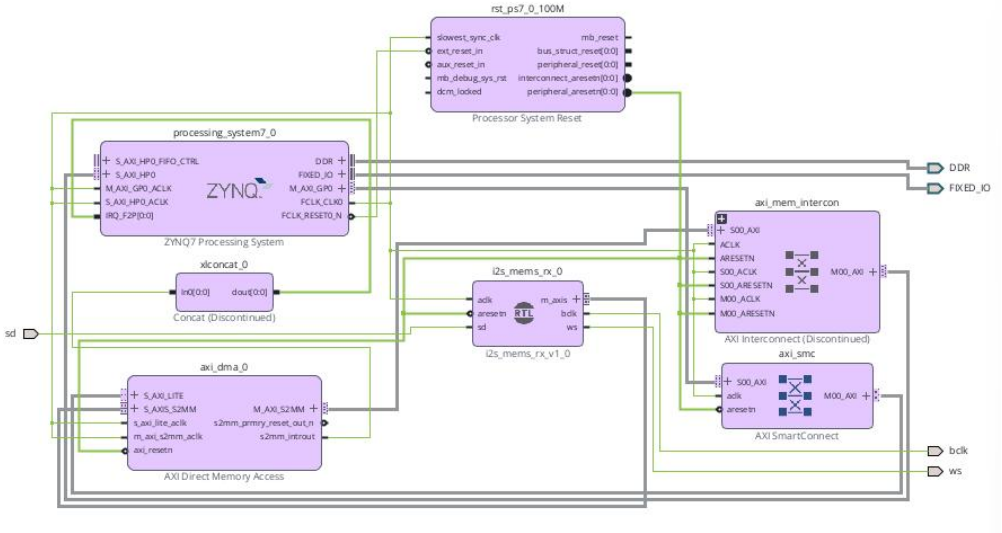


图 3-5 音频采集模块 PL 设计

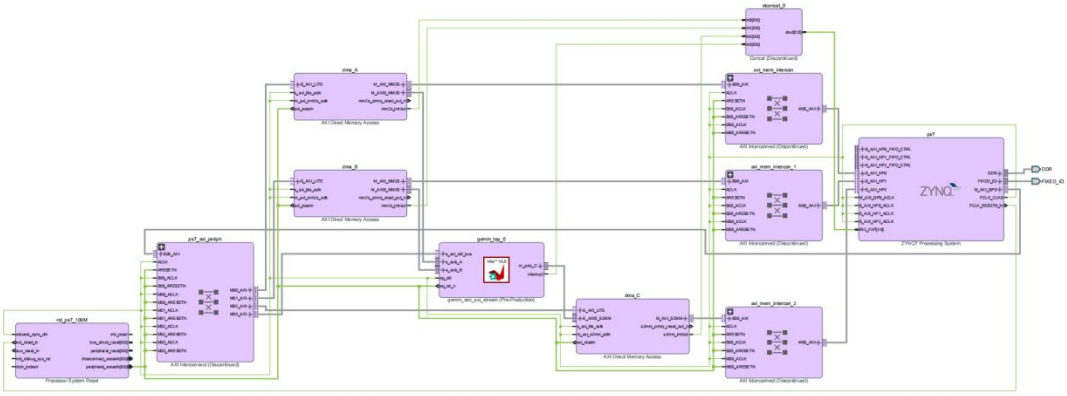


图 3-6 GEMM 加速模块 PL 设计

3.2.3 软件成果

软件由板端运行程序与模型（src/ps 的 A 类文件）、7 个部署与验证脚本（board/）、6 个通用验证工具（skill/tools）组成，以 pitta-audio、pitta-vision 两个 systemd 服务常驻运行。控制台实时输出鸣声分数、状态机与推理耗时；运行产物写入 runtime/annotated（快照与 JSON 预测日志）与 runtime/videos（H.264 MP4）。

```
xilinx@pynq:~$ sudo bash /home/xilinx/start_pitta_audio.sh
[sudo] password for xilinx:
[BOOT] pl_server 已就绪（第 1 次检查）
[MIC] Listening; saved WAV time is received-sample time.
[OVERLAY] inmp441.bit 已加载；FCLK0=100.0 MHz；DMA=axi_dma_0
[MODEL] Warm-up 0.342s; input 32000 Hz / 2s
[GATE] threshold=0.850 min_hits=1 release=0.300/2 windows cooldown=5.0s
[AUDIO] 0.00~2.00s score=0.0021 rms=-45.2dBFS infer=0.131s wait=0.002s skipped=0
[AUDIO] 1.50~3.50s score=0.0043 rms=-43.8dBFS infer=0.128s wait=0.001s skipped=0
[AUDIO] 3.00~5.00s score=0.0018 rms=-46.1dBFS infer=0.130s wait=0.002s skipped=0
[AUDIO] 4.50~6.50s score=0.0029 rms=-44.5dBFS infer=0.129s wait=0.002s skipped=0
[AUDIO] 6.00~8.00s score=0.0036 rms=-45.0dBFS infer=0.130s wait=0.001s skipped=0
[AUDIO] 12.00~14.00s score=0.9921 rms=-21.4dBFS infer=0.130s wait=0.002s skipped=0 TRIGGER
[BRIDGE] published event=a1b2c3d4e5f6a7b8
[鸟鸣时间] 北京时间=2026-11-02T16:12:40.133+08:00 | Unix=1793607160.133124 | score=0.9921 | 录音窗口=12.00~14.00s | event_id=a1b2c3d4e5f6a7b8
[AV] PAUSED: recording/conversion; microphone WAV continues.
[AV] RESUME: waiting for 2 seconds of fresh audio.
[AUDIO] 18.00~20.00s score=0.0052 rms=-42.9dBFS infer=0.128s wait=0.001s skipped=0
```

图 3-7　软件运行截图（音频部分）

```
xilinx@pynq:~$ sudo bash /home/xilinx/start_pitta_vision.sh
[sudo] password for xilinx:
[BOOT] 已找到 /dev/video0（第 1 次检查）
[CAM] 请求 640x480 @5 FPS MJPG
[CAM] 实际 /dev/video0: 640x480 格式=MJPG 驱动报告FPS=5.00
[INFER] 常驻推理进程已启动(模型只加载一次)
[STATUS] WAITING；采集5.0 FPS；Ctrl+C退出
>>> 收到声音触发，开始搜鸟(30s窗口)
[STATUS] SEARCHING；采集5.0 FPS
[INFER] 帧26 推理5.820s，采集FPS=5.00，结果帧距今0.10s
[检测] 发现仙八色鸫! conf=0.91 框数=1 (单帧耗时5.8s)
[AV] Audio prediction paused during recording/conversion.
>>> 开始录像: pitta_1793607160133.mp4
[STATUS] RECORDING；采集5.0 FPS
[RECORD-INFER] bird=True misses=0 span=0.00s infer=5.90s
[RECORD-INFER] bird=True misses=0 span=5.80s infer=5.85s
[RECORD-INFER] bird=False misses=1 span=0.00s infer=5.78s
[VIDEO] Stop: confirmed_absence; recorded wall time 12.34s
[VIDEO] 录像结束，正在转换H.264 MP4；请等到“MP4保存完成”再复制文件。
[VIDEO] MP4保存完成：/home/xilinx/runtime/videos/pitta_1793607160133.mp4 (62帧，12.40s，H.264/yuv420p)
[AV] Recording/conversion finished; audio may resume.
>>> 录像保存: pitta_1793607160133.mp4 (62帧)
[STATUS] WAITING；采集5.0 FPS
```

图 3-8　软件运行截图（图像部分）

```
xilinx@pynq:~$ cp /home/xilinx/audio_model.npz /home/xilinx/bird_sample.wav /home/xilinx/board/
xilinx@pynq:~$ cd /home/xilinx/board
xilinx@pynq:~/board$ ls *.npz *.wav
audio_model.npz  bird_sample.wav
xilinx@pynq:~/board$ sudo python3 e2_board.py --bench 20
[sudo] password for xilinx:
[音频] bird_sample.wav  sr=32000  采样点=64000
[GEMM] M=12864 K=9 N=16  特征图=64×201
[HW] GEMM overlay 就绪: M=12864 K=9 N=16, DMA C 上限=8388608 字节
======== 公平加速比（板端，各 20 次，中位数）========
硬件 conv1 端到端（DMA搬运+GEMM+反量化；量化循环外只做一次）   = 0.0527 s
其中 HLS 核 ap_start→ap_done 纯计算                          = 0.0067 s
numpy conv1 单独（A@B+bias+ReLU）                             = 0.0310 s
公平加速比 = 0.59x（端到端口径）
纯核加速比 = 4.64x（仅核计算，不含搬运）
最小/最大 硬件端到端 = 0.0526 / 0.0528 s
================================================
```

图 3-9　软件运行截图（加速部分）

3.3 特性成果

功能与性能的量化结果如表 3-1、表 3-2。

表 3-1 功能验证结果

序号	测试项	测试方法	结果
1	声学监听与触发	9 段正样本、155 个人耳核对事件	事件级召回 97.4%（阈值 0.985）
2	抗误触发	18 段负样本、2063 个窗口	0 误报，最高分 0.0329
3	视觉确认	266 张独立网络图	图像级召回 96.2%（256/266）
4	视觉特异性	380 张负样本（321 其他鸟 + 59 背景）	380/380 无虚警
5	存在性录像	端到端联动日志	37.40 s / 187 帧，缺席 3 次停录
6	预测日志	检查 runtime/annotated	frame_*.json 含时间戳/框/置信度/耗时
7	开机自启	systemd 托管	两个服务均开机自启、异常拉起
8	连续运行	stability_monitor.sh 挂 ≥8h	≥8h 连续运行，服务无异常重启

表 3-2 性能测试结果

指标	测试方法	结果
音频单窗推理	板端单窗口计时	2.075 s
图像单帧推理	ncnn 320×320、2 线程	5.65 – 6.72 s
声到像触发延迟	联动日志时间戳	约 10.5 s
录像后滚时间	缺席判定跨度	约 11.9 s
conv1 纯核	e2e_board.py --bench 20，中位数	6.7 ms
conv1 端到端	DMA+GEMM+反量化，20 次	52.7 ms
numpy conv1	A@B+bias+ReLU，20 次	31.0 ms
公平加速比	端到端 / 纯核	0.59× / 4.64×



图 3-10 现场测试照片

测试条件：室内安静环境，声源距麦克风约 70 cm；视觉测试含训练池与独立测试两套口径；板卡室温、Micro USB 5 V 供电；时延类指标除注明外均为单次或小样本测量，非基准测试。

3.4 优化过程与前后对比

研发按“测量—定位—修正—回归”推进：每个缺陷都先复现现象、再由日志或仿真定位根因，修正后用可执行的检查回归（C 仿真断言、hwh 自检、Overlay 加载）。关键修改见表 3-3，可量化的前后对比见表 3-4。第 1—4 项解决“不能用”，第 5 项解决“值不值”——前者是工程可用性，后者是设计取舍。

表 3-3 问题与修正记录

序号	问题现象	原因	修正	验证方式
1	全量矩阵乘报超过 16384 字节	DMA 长度寄存器仅 14 bit	改为 23 bit（8 MB），保持直通	hwh 自检 C_SG_LENGTH_WIDTH=23
2	S2MM 停在 Running，IOC 不置位	HLS 未拍 TLAST 在流水线排空时丢失	改用跨 tile 单调递增的全局包计数	C 仿真逐包 TLAST 断言 PASS
3	PYNQ 实例化 DMA 报 KeyError	xlconcat 先建 4 口使 NUM_F2P=4	先建 1 口锁定参数再扩口	Overlay 正常加载

序号	问题现象	原因	修正	验证方式
4	BD 校验报 m_axi_sg_aclk 未连接	开启 SG 引入新时钟域	关闭 SG，改直通模式	BD 校验通过
5	conv1 上硬件后整体反而变慢	DMA 搬运占端到端 87%	主程序保留 numpy，GEMM 作能力验证	--bench 20 实测

表 3-4 优化前后对比

指标	优化前	优化后	变化	依据
DMA 单次传输上限	16 KB（14 bit）	8 MB（23 bit）	提升 512 倍	gemm_soc.hwh
S2MM 完成标志	不置位，停在 Running	正常置位，DMA 可反复调用	从不可用变为可用	C 仿真 TLAST 断言
Overlay 加载	KeyError，无法实例化	正常加载 5 个 IP	从不可用变为可用	板端 ip_dict
conv1 纯核耗时	numpy 31.0 ms	硬件 6.7 ms	4.64 倍	--bench 20 中位数
conv1 端到端耗时	numpy 31.0 ms	硬件 52.7 ms	0.59 倍（变慢）	--bench 20 中位数
conv1 数值一致性	—	最大误差 0.001838	205824 点 0 超差	--bench 20 中位数

加速后反而变慢的原因在于：conv1 的计算量只有 12864×9×16 ≈ 185 万次乘加，而一次调用需要在 DDR 与 PL 之间搬运 A（231.6 KB）、B（0.3 KB）、C（411.6 KB），合计约 628 KB，算术强度约 2.9 次乘加/字节，属于典型的访存受限任务。按端到端 52.7 ms 折算，有效搬运速率仅约 12 MB/s，远低

于 32 位 HP 口的能力，说明瓶颈不在带宽，而在每次调用的固定开销：重新配置 DMA、对缓冲区做 cache 刷新与失效、以 0.5 ms 粒度轮询状态位，以及缺少双缓冲与多包流水。相比之下，numpy 直接在同一次推理已驻留 cache 的数据上运算，没有这次往返。

换言之，FPGA 的优势在持续流式、可流水的负载；对一次只有 1.85M 乘加的短任务，启动与搬运开销占了端到端的 87%，无法摊销。这也是正式主程序保留 numpy、只把 GEMM 作为加速能力验证的原因——不是核做得不对，而是负载与器件不匹配。

资源上，两块设计合并是可行的：采集 overlay 占 3.15% LUT、0 个 DSP，加速 overlay 占 10.11% LUT、30.45% DSP，合并后约 13.3% LUT，DSP 仍为 30.45%（采集设计不含 DSP）。当前保持分时加载的真正原因是二者用途不同——采集 overlay 用于实时监听、常驻；加速 overlay 只用于能力验证、按需加载，无需同时驻留。

第四部分　总结

4.1　可扩展之处

（1）合并两块 overlay：把 I^2S 采集与 GEMM 放进同一 Block Design，可省去分时加载；但需先解决 DMA 搬运瓶颈，否则端到端收益仍有限。

（2）扩核算子：将 K 方向分块累加引入 GEMM 核，可覆盖 conv2/conv3（K=144/288），才可能改变端到端加速比。

（3）物种扩展：替换 audio_model.npz 与 best_ncnn.*，并调整阈值即可；采集核、DMA 与录像状态机全部复用。

（4）部署扩展：加 4G/太阳能与防护外壳，记录文件按容量滚动清理，可支持野外长期布设。

4.2　心得体会

这个作品最初的想法很简单：让板子听见鸟叫就看看有没有鸟。真正做下来才发现，难点几乎都藏在数据通路上。

第一个坑是 TLAST。末拍标志原本用最后一个分块的最后一行判断，仿真

正常，上板后 S2MM 却一直停在 Running。定位到输出流水线排空阶段下标回绕导致标志丢失，改成跨分块单调递增的全局包计数后解决。这让我们第一次真切体会到，逻辑上等价和时序上等价是两回事。

第二个坑是 DMA 长度。三个 DMA 都报超过最大传输长度 16384 字节，我们一度以为没开 Scatter-Gather；翻手册后才发现是长度寄存器只有 14 位。改成 23 位后，8 MB 单次传输一次通过。教训是：报错描述的往往只是现象，根因要去寄存器定义里找。

第三个坑来自工具链。PYNQ-Z2 是 32 位 ARM，PyTorch 装不上，只能把模型导出成 numpy 权重，视觉端改用 ncnn 并在板端编译 C++ 推理程序。过程枯燥，但让我们看清了每层的计算量与访存。

最意外的是第四个坑：把 conv1 搬上 PL 之后，端到端反而更慢——纯核 6.7 ms、比 numpy 快 4.64 倍，但 DMA 搬运把收益吃光，端到端只有 0.59 倍。我们没有硬说加速成功，而是把这个结论写进文档，并据此让正式主程序继续走 numpy。这件事让我们明白，软硬划分不是看哪个算子算得快，而是看数据搬得动搬不动。

在协作上，我们把板端能跑拆成一条条可验证的小步骤：先只加载 overlay 打印 IP 列表，再单独跑 GEMM 比对数值，最后才接麦克风和摄像头。每次只改一个变量，出错时就能立刻定位。

第五部分　参考文献

[1] Bota G, Manzano-Rubio R, Catalán L, et al. Hearing to the unseen: AudioMoth and BirdNET as a cheap and easy method for monitoring cryptic bird species[J]. Sensors, 2023, 23(16): 7176.

[2] Kahl S, Wood C M, Eibl M, et al. BirdNET: A deep learning solution for avian diversity monitoring[J]. Ecological Informatics, 2021, 61: 101236.

[3] Blum D, Häring E, Jirges F, et al. Zwitscherkasten: DIY audiovisual bird monitoring[EB/OL]. arXiv:2602.13330, 2026.

[4] Hervás M, Alsina-Pagès R M, Alías F, et al. An FPGA-based WASN for remote real-time monitoring of endangered species: a case study on the birdsong recognition

of Botaurus stellaris[J]. Sensors, 2017, 17(6): 1331.

[5] 国家林业和草原局, 农业农村部. 国家重点保护野生动物名录[Z]. 2021.

[6] Tencent. ncnn: a high-performance neural network inference framework[CP/OL]. https://github.com/Tencent/ncnn.

[7] Xilinx. Vitis High-Level Synthesis User Guide: UG1399[R]. 2023.

[8] Jocher G, Chaurasia A, Qiu J. Ultralytics YOLO (Version 8.0)[CP/OL]. 2023. https://github.com/ultralytics/ultralytics.

[9] Xilinx. PYNQ: Python productivity for Zynq[EB/OL]. https://www.pynq.io.

[10] Xilinx. AXI DMA v7.1 LogiCORE IP Product Guide: PG021[R]. 2022.

[11] BirdLife International. Pitta nympha. The IUCN Red List of Threatened Species[EB/OL]. https://www.iucnredlist.org.

附录 A　大模型协作记录

本附录完整落地 3 个协作案例（提示词、模型回答、验证、沉淀），完整记录（24 bit 解码错误、子线程 signal 报错、录像逻辑三次迭代等）见仓库 report/llm_collaboration.md。

A.1　案例一：AXI DMA S2MM 寄存器偏移

现象：S2MM 通道不传输，向 0x00 偏移写 CR 无任何反应。

提示词（节选）：“AXI DMA 的 S2MM 通道寄存器偏移是多少？我用的是 Xilinx AXI DMA 7.1 IP，只使能了 S2MM 通道。我在 .hwh 文件里看到基地址是 0x40400000，但写 0x00 偏移的 CR 寄存器不生效。”

模型回答（关键纠错）：S2MM 通道的寄存器并不从 0x00 开始——MM2S 为 CR=0x00、SR=0x04、DA=0x18、LEN=0x28，S2MM 为 CR=0x30、SR=0x34、DA=0x48、LEN=0x58；只使能 S2MM 时应从 0x30 开始访问。

验证：修改偏移后 DMA 正常传输，后续驱动与诊断脚本统一使用该偏移表。

沉淀：不再凭记忆写偏移，改为用 hwh_probe.py 从 .hwh 生成寄存器表并对账源码（技能包工具之一）。

A.2　案例二：INMP441 数据全 0

现象：DMA 能收到数据，但左右声道 RMS 都接近 0。

提示词（节选）：“INMP441 I^2S 麦克风数据全 0，可能是什么原因？我已经确认 BCLK/WS/SD 引脚约束正确，DMA 也能收到数据但全是 0。L/R 引脚我没接，悬空了。”

模型回答（关键纠错）：L/R 引脚必须接 GND 或 VDD，不能悬空；接 GND 选左声道、接 VDD 选右声道，悬空时内部状态不确定，可能输出全 0；建议用万用表蜂鸣档确认 L/R 到 GND 的导通性。

验证：万用表确认 L/R 确实未接地，接地后左声道 RMS=753545，数据恢复正常。

沉淀：把 L/R 写入接线检查表，成为上板前的固定检查项。

A.3　案例三：GEMM 上板——S2MM 不置 IOC 与 16 KB 上限

现象一：recvchannel.transfer(bufC) 返回后 DMASR 的 IOC 位始终不亮，缓冲里已有数据，通道却停在 Running。

提示词（节选）：“AXI DMA S2MM 通道 transfer 之后 IOC 位一直不置位，但 buffer 里有数据了。HLS 核输出是 AXI-Stream，可能是什么原因？”

模型回答（关键纠错）：HLS 生成的 stream 在最后一个 beat 没有拉 TLAST，而 S2MM 靠 TLAST 判断一帧结束；修法一是让 HLS 末拍显式拉 TLAST，修法二是旧 bit 兼容——不以 S2MM 的 IOC 为完成判据，改用 HLS 核 ap_ctrl 的 ap_done（bit1）作为计算完成的权威判据，缺 IOC 时通过 stop→sleep→start 复位通道复用。

验证：gemm_board.py 改为轮询 ap_done 判定完成、缺 IOC 时复位通道，硬件可反复调用。

现象二：C 缓冲 411 KB，DMA transfer 直接失败。

提示词（节选）：“PYNQ AXI DMA recvchannel._size 报 16384 字节，但我需要传 411 KB。怎么提高单次传输上限？”

模型回答：这是 AXI DMA IP 的 c_sg_length_width 参数，默认 14（2^14

= 16 KB）；在 Vivado 中改为 23（2^23 = 8 MB）重新综合即可，板端可用 dma.recvchannel._size 验证是否变为 8388608。

验证：重新综合后 _size=8388608；板端脚本增加预检，容量不足时直接报错并提示重新覆盖 bit/hwh。

沉淀：两条经验分别写成技能包的 D1（逐包断言 TLAST）、D2（先查长度位宽再查 SG）。

A.4 采纳与未采纳

模型给出的建议并非全部采纳，每条都经过可执行验证后才落地。

表 A-1 采纳内容、验证方式及理由

建议内容	验证方式	采纳理由
S2MM 寄存器偏移从 0x30 开始，而非 0x00	写 0x30 后 DMA 正常传输	hwh 文件中基址已确认，纠正了“MM2S/S2MM 同基址”的直觉错误
INMP441 的 L/R 引脚不能悬空	接地后左声道 RMS 从 0 恢复到 753545	万用表量通断后确认，模型提醒了数据手册未强调的细节
DMA 长度位宽改为 23 bit	板端 _size 从 16384 变为 8388608	重新综合后一次通过
缺 IOC 时以 ap_done 为完成判据	gemm_board.py 可反复调用	过渡方案，后续 TLAST 修正后已不再依赖

表 A-2 未采纳内容及理由

建议内容	未采纳理由
开启 Scatter-Gather DMA 以提升带宽	开启后引入新时钟域导致 BD 校验失败；直通模式已满足 256 kB/s 音频流需求
接入 DMA 中断替代轮询	PYNQ 32 位镜像下中断配置复杂，0.5 ms 轮询周期对 2 s 窗推理无影响，风险大于收益
将 conv2、conv3 也映射到 PL	当前核 K≤16，需重做分块；且 conv1 端到端仅 0.59×，瓶颈在搬运不在计算
用 dma.recvchannel.transfer() 高级封装	默认按 16 KB 分包，隐藏了包边界与 TLAST 控制；本项目需精确时序，直接 MMIO 更可控

A.5　协作方式与沉淀

协作固定为三步：先给约束（接口、位宽、时序、板卡型号），再由模型给实现，最后用可执行的自检验收——C 仿真是否 PASS、hwh 中位宽是否为 23、MD5 是否与电脑端一致、寄存器回读是否正确；任一项不过就带原始报错回到上一步。

过程中确立三条口径：真值不能用模型自身输出代替；加速比必须同板同口径（因此 6.7 ms 的纯核耗时不能直接除以 numpy 整网 0.497 s）；估计值必须标注为估计。这三条已写入 skill/SKILL.md 的判定口径。

上述经验沉淀为可复用的 PYNQ 验证技能包（附录 B），6 个工具与 3 份模板均只依赖 Python 标准库，可离线运行。

A.6 协作心得

与 AI 进行多轮交互后，最大的感受是：AI 擅长在短时间内列出可能性、翻手册、给出候选方案；但是方案对不对、值不值得做，需要人工上板验证；验证结果说明什么问题，也需要自己思考总结。

最典型的例子是 conv1 加速。模型给出的 HLS 代码在 C 仿真下数值完全正确，纯核比 numpy 快 4.64 倍 —— 当时觉得这就是很好的成果了。但板上端到端一测只有 0.59×，比 numpy 还慢，DMA 搬运占了 87%。这个结果并不是模型能预判的，必须靠人把纯核和端到端分开测、把瓶颈归因到算术强度和 cache 行为。最终我们没有硬说加速成功，而是把这个结论写进报告，这反而成了很有说服力的工程判断。

当然 AI 也有出人意料地靠谱的时候。TLAST 丢失那个问题，我们自己查了两天 testbench 都没定位到是流水线排空阶段下标回绕；AI 直接指出“stream 末拍没拉 TLAST”，我们恍然大悟。

后来我们固定了一个节奏：在模型给出的方案中，我们会认真挑选一个最可能的上板尝试，不对就带着报错回去问。不盲目相信 AI，也不把它当搜索引擎用 —— 它最大的价值不是给出标准答案，而是帮助我们快速缩小问题排查范围。

附录 B　技能包的提炼过程

技能包并非先设框架后找场景，而是从本项目反复出现的故障中逐条提炼而成。内容位于仓库 skill/ 目录，包括 6 个工具、3 份模板，另附 pitfalls 故障清单与 SKILL.md 说明文件。

B.1 失败与观察的来源

六个工具分别对应一类已实际发生的故障。

hwh_probe.py 对应寄存器偏移误写。该工具从 .hwh 解析实例、基址与寄存器表，支持以 --emit-driver 生成驱动骨架，并以 --audit 比对源码中的硬编码偏移与 .hwh 的一致性。

bd_lint.py 对应“块图校验通过而板端异常”。该工具静态检查 Tcl、.hwh 与 .xdc 三者的一致性及 AXI 位宽配置；退出码为 1 表示存在 ERROR，须修正后再进入综合。

dma_selftest.py 对应“DMA 不启动或数据全为 0”。使用时应先以 --dry-run 确认通道、偏移与测试项，再在板端执行；判定依据为覆盖字数大于 0 且错误位为 0，而非 wait() 是否返回。

vivado_report.py 对应“修改后无法判断优劣”。该工具从 utilization 与 timing 报告中提取资源与时序指标，支持以 --baseline 接入上一版结果，用于回归对比。

golden_compare.py 对应“硬件输出与参考不一致”。该工具按容差比对，本项目采用 int16 与 Q12 定标，容差据此校准。

verify_pack.py 负责最终收口，将上述检查汇总为 verification/report.md。

三份模板亦来自具体需求：hw_sw_partition_prompt 用于确定算子放置位置；trigger_fsm 用于事件触发的多进程状态机设计；presence_recording 用于实现目标存在性录像。

B.2 有效性的验证

验证分为离线与板端两个层次。六个工具均提供 --self-test，无需 Vivado 与板卡即可确认工具自身可用；板端结论由 verify_pack.py 统一生成，报告中的每一项数据均可由命令复现，无法复现的估计值须明确标注。

需要说明的是，报告中的 10 项检查均属离线检查，不等同于板端验证。只有在 board_checklist.md 中逐项回填并保留板端原始输出后，方可视为完成验证。

B.3 边界的确定

以下 8 条失效条件均来自实际失效后的补充，而非预先设定：

1. 目标平台非 Xilinx Zynq 或缺少 .hwh 文件——寄存器偏移与位宽均依赖 hwh，此类工程不适用；

2. .bit 与 .hwh 非同一版本——寄存器映射与硬件不一致，读取到的“正确值”亦不可信；须由同一版工程同时导出，并先做 MD5 与文件大小核对；

3. 仅提供 .xsa 而无 .hwh——hwh_probe 不解析 xsa，须先导出 hwh；

4. 接口非 AXI DMA（VDMA、AXI-Stream FIFO 或自定义 DMA）——dma_selftest 的寄存器流程按 AXI DMA 实现，须修改映射或不适用；

5. 精度或定标发生变化——容差按 int16 与 Q12 校准，须重新确定容差并重跑黄金比对；

6. 模板运行于 NFS、容器只读盘或 Windows 共享目录——触发与录像模板依赖同一文件系统内的 os.link() 原子发布，此类环境不成立；

7. PYNQ 大版本升级（如 2.x 至 3.x）——Overlay、Clocks、allocate 的行为可能变化，须先执行 verify_pack 与 dry-run；

8. 结论未经板端执行——视为未验证。

判定口径共 4 条：寄存器可读不代表通道存在，偏移以 .hwh 为准；传输完成不代表数据到位，须同时提供覆盖率与错误位证据；时序结论以 timing.rpt 的结论句与 WNS 为准，不以综合通过替代；报告中每一项数据须可由命令复现。

技能包与具体题目解耦。更换题目或更换 Zynq 板卡后，只要仍可导出 .hwh 且使用 PYNQ 的 Overlay 与 DMA，6 个工具与 3 份模板即可继续使用；与项目绑定的仅是本项目的具体偏移与容差。

附录 C 复现说明

本附录给出从零复现的完整步骤，按硬件连接、构建、部署、运行、验证五个阶段组织。

C.1　环境与目录

PC 端使用 Vivado 2026.1 与 Vitis HLS 2026.1；板端为 PYNQ-Z2 官方镜像，Python 3 及 numpy、OpenCV、ncnn 由 setup.sh 统一安装。

仓库目录与文件名全部为英文，中文仅出现在正文与文档中：

pitta-bird-monitor/

├──build/　构建脚本与产物（build.tcl、build_gemm.tcl、inmp441.bit/.hwh、gemm/gemm_soc.bit/.hwh、utilization/timing/power 报告）

├── src/fpga/　I^2S 接收核 RTL、引脚约束、GEMM HLS IP

├── src/ps/　板端程序、模型与 PC 端训练/评估脚本（A 类为板端部署文件）

├── board/　启动脚本、systemd 服务、GEMM 验证与长测脚本

├── data/　测试音频、测试图像、GEMM 测试向量（a/b/c_ref.bin）、comparison.csv

├── skill/　PYNQ 验证技能包（6 工具、3 模板）

├── sim/　I^2S 核 testbench

└── report/　设计报告、作品简介、大模型协作记录、海报

C.2　部署与运行

表 C-1　从零复现步骤

步骤	操作	关键命令/检查点
1	硬件连接	INMP441→PmodB 针 1/2/3（W14/Y14/T11），VDD 3.3 V，L/R→GND
2	烧录启动	PYNQ-Z2 镜像写入 MicroSD，网线接入 192.168.2.99
3	上传运行文件	scp A 类文件（见 src/ps/README.md）到 /home/xilinx
4	配置环境	sudo bash setup.sh（依赖、runtime 目录、摄像头权限、编译 infer_ncnn）
5	正式运行	启用 pitta-vision.service 与 pitta-audio.service，journalctl -f 观察
6	GEMM 验证	收集 build/gemm 与 src/ps 到 /home/xilinx/gemm，bash verify_gemm.sh 后跑 e2e_board.py
7	留存证据	拷贝 runtime/annotated、runtime/videos 与 journal 日志

C.3　重建 PL 与模型

PL 侧在 PC 上运行两个 Tcl 脚本：build/build.tcl 产出 inmp441.bit 与 inmp441.hwh，build/build_gemm.tcl 产出 gemm_soc.bit 与 gemm_soc.hwh，两者均由同一版工程同时导出，避免 bit 与 hwh 不配套。

HLS 侧对 gemm.cpp 依次执行 C 仿真、C 综合、C/RTL 联合仿真与 IP 打包，输出至 src/fpga/gemm。

模型侧：视觉模型由 src/ps/03_train.py 训练、04_export.py 导出 ncnn；声学模型由 src/ps/train_binary_fixed.py 训练并导出 audio_model.npz。

C.4　常见故障与处理

1. DMA 报“超过最大传输长度 16384 字节”： DMA 长度寄存器为 14 bit，应重新综合为 23 bit（8 MB），覆盖 bit/hwh 后重启；

2. S2MM 停在 Running、IOC 不置位：HLS 末拍缺少 TLAST，应以核的 ap_done 作为完成判据，并在缺 IOC 时复位通道；

3. transfer 后 wait 不返回、单核占用 100%：DMA 中断未接入 PS，应设置 PYNQ_DISABLE_DMA_INTERRUPT=1 并改为轮询；

4. 采集数据全为 0：INMP441 的 L/R 悬空，应将 L/R 接 GND（左声道）或 3.3 V（右声道）；

5. 音频服务约每 30 分钟重启一次，每 1800 s 主动换会话，属设计行为，非故障，节点可长期无人值守运行。